

Reviewer Pack — Polymatroid Rank Lower Bound for Monotone DNF Representing k...

Entry c3ca2e711eef · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 01:18:30 UTC

Polymatroid Rank Lower Bound for Monotone DNF Representing k-CLIQUE

Entry ID: c3ca2e711eef **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 01:18:30 UTC

1. Conjecture

Field A (mathematical branch): Polymatroid Theory **Field B** (complexity object): Monotone DNF Size

Statement:

For any k-CLIQUE indicator function on n vertices, its minimal monotone DNF size is $\Omega(\rho(n,k)^2)$, where $\rho(n,k)$ is the rank of the polymatroid defined by the intersection lattice of k -element subsets. For random 3-CNF formulas with clause-density 4, the polymatroid rank $\rho(n,k) \leq \log n$ implies DNF size $\geq n^{\{1.5\}}$.

Rationale (proposer's reasoning):

Polymatroid rank captures the combinatorial geometry of inclusion-exclusion constraints in DNFs. By linking it to k-CLIQUE's lattice structure, we expose inherent complexity in representing clique indicators, which are known to resist efficient monotone circuit representations.

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e30f0693c3a81062

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def gaussian_elimination(A, b):
    n = len(b)
    augmented_matrix = [A[i] + [b[i]] for i in range(n)]

    # Forward elimination
    for i in range(n):
        if augmented_matrix[i][i] == 0:
            return None # Singular matrix

        for j in range(i+1, n):
            factor = augmented_matrix[j][i] / augmented_matrix[i][i]
            for k in range(n + 1):
                augmented_matrix[j][k] -= factor * augmented_matrix[i][k]

    # Backward substitution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = augmented_matrix[i][-1]
        for j in range(i+1, n):
            x[i] -= augmented_matrix[i][j] * x[j]
        x[i] /= augmented_matrix[i][i]

    return x

def rank(matrix):
    A = [row[:] for row in matrix]
    b = [0] * len(A)
    rref = gaussian_elimination(A, b)
    if rref is None:
        return 0
    rank = sum(1 for row in rref if any(row[i] != 0 for i in range(len(row)-1)))
    return rank

def generate_3cnf(n, clause_density):
```

```

num_clauses = int(n * n * clause_density / 2)
variables = list(range(1, n+1))
clauses = set()

while len(clauses) < num_clauses:
    a, b, c = random.sample(variables, 3)
    if (a, b, c) not in clauses and (a, c, b) not in clauses and (b, a, c) not in clauses \
        and (b, c, a) not in clauses and (c, a, b) not in clauses and (c, b, a) not in clauses:
        clauses.add((a, b, c))

return clauses

def dnf_size(clauses):
    return len(clauses)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 20
    clause_density = 4
    num_trials = 30

    total_dnf_size = 0
    count_supports_conjecture = 0
    counterexample_found = False

    for _ in range(num_trials):
        clauses = generate_3cnf(n, clause_density)
        dnf_size_value = dnf_size(clauses)

        if len(clauses) == 0:
            continue

        rho_n_k = rank([[1 if i in clause else 0 for i in range(1, n+1)] for clause in clauses])
        if rho_n_k is None or rho_n_k <= math.log(n):
            total_dnf_size += dnf_size_value
            if dnf_size_value >= n ** 1.5:
                count_supports_conjecture += 1

    mean_dnf_size = total_dnf_size / num_trials
    support_fraction = count_supports_conjecture / num_trials

    return {
        "metric_name": "DNF Size",
        "metric_value": mean_dnf_size,
        "instances_tested": num_trials,
        "conjecture_holds": support_fraction >= 0.8,
        "counterexample": "" if counterexample_found else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 53)) # Default to first 30 primes

    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_dnf_size = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_dnf_size} std=0.0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b8682b4f.py", line 106, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b8682b4f.py", line 83, in run_trial
    rho_n_k = rank([[1 if i in clause else 0 for i in range(1, n+1)] for clause in clauses])
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b8682b4f.py", line 44, in rank
    rref = gaussian_elimination(A, b)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b8682b4f.py", line 29, in gaussian_elimination
    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
    ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError during rank computation, preventing data collection | next:
Fix the gaussian_elimination implementation to handle matrix dimensions properly

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	35097
2	preregistration	ollama_remote	qwen3:8b	0	0	20071
3	novelty	ollama_remote	qwen3:8b	0	0	16532
4	novelty	ollama_remote	qwen3:8b	0	0	10968
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18380
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11601
7	judge	ollama_remote	qwen3:8b	0	0	12728

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 125377 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/c3ca2e711eef.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c3ca2e711eef.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c3ca2e711eef.tar.gz` (if generated)